

增強課程 4：Implicit Enhancement Point (Source Code Plugin) 實戰——客製化工單自動給號

Lecture

前面三題都是「SAP 官方預留插入點」的擴充技術——**SMOD/CMOD** 的 Function Exit、Classic BAdI 都要求原開發者事先在標準程式裡宣告一個具名的插入點 (**CALL CUSTOMER-FUNCTION**、**GET BADI**)。這題要講的 **Implicit Enhancement Point** (隱式增強點) 完全不同：它不需要原開發者事先宣告任何東西——任何 **FORM/METHOD/FUNCTION** 的開頭、結尾，幾乎每個陳述句之間，系統都自動留了一個隱式插入點，只是平常編輯器不會顯示出來。

Implicit 與 Explicit 的關鍵差異：Explicit Enhancement (**ENHANCEMENT-POINT/ENHANCEMENT-SECTION**，本課程 en06 主題) 要原開發者主動寫一行宣告，才會出現插入點；Implicit 則是框架保證到處都有，代價是只能插入程式碼、不能改變介面 (不能加參數、不能改簽章)，而且同一個插入點可以同時掛多個 **Source Code Plugin**，執行順序依 Enhancement 建立順序疊加。

真實案例背景：這題的藍本是一個真實生產環境的客製化——標準 **FM CO_ZF_NUMBER_GET** (Function Group **COZF**，套件 **CO**，SAP 標準物件，是生產訂單 / 工單建立時決定訂單號碼 **CAUFVD-AUFNR** 的核心邏輯) 在**最前面** (所有標準程式碼之前) 插入一段客製化邏輯：如果這個工單的廠別 + 製令類型組合有登記要用自訂編號，就依「Lead Code + 年月 + 流水號」的格式產生號碼，取代標準 Number Range 給的號碼；否則什麼都不做，讓標準邏輯正常執行。

為什麼插在「最前面」不會被後面的標準程式碼蓋掉——關鍵是 **EXIT**：

```
ENHANCEMENT 1 ZEN04_ORDER_NO_PLUGIN.  
  IF object = 'AUFTRAG'.  
    " ... 查兩層自訂規則表 ...  
    IF lv_applicable = abap_true.  
      caufvd_exp-aufnr = lv_aufnr.  
      EXIT. "← 關鍵：立刻結束整個 FUNCTION  
    ENDIF.  
  ENDIF.  
ENDENHANCEMENT.
```

EXIT 寫在 **FUNCTION/FORM/METHOD** 主體裡，效果跟 **RETURN** 一樣，會立刻結束這個常式。條件成立時，**EXIT** 讓 FM 的其餘標準程式碼 (**DATA:** 宣告、**CALL FUNCTION 'NUMBER_GET_NEXT'**、**CASE OBJECT.** 判斷) **完全沒有機會執行**，不是「執行完再被覆蓋」，是「整段被跳過」；條件不成立 (廠別/製令類型沒登記、或規則表查無資料)，Enhancement 什麼都不做，程式正常往下走到標準邏輯——這是**兩層安全閘**：第一層是「廠別 + 製令類型是否登記啟用」，第二層是「規則表是否真的查得到資料」，任一層沒過就 fall through 走標準取號，不會用空值硬闖。

設計：三張自訂表，職責分離：

表名	Key	資料欄位	用途
ZEN04_PLTAUART	WERKS+AUART	(無，純存在性判斷)	啟用總開關——這個廠別/製令類型組合要不要用自訂編號
ZEN04_RULE	WERKS+AUART+FEVOR+ZGRTYPE	LEADCODE、STNUM	依排程群組 + 收貨對象決定編號前綴與起始流水號
ZEN04_SEQ	WERKS+AUART+FEVOR+ZGRTYPE+LEADCODE+ZYEAR+ZMONTH	NUMNO、AUFNR	每個年月各自的流水號進度，第一次自動 INSERT，之後自動 UPDATE 遞增

ZEN04_RULE / ZEN04_SEQ 的 FEVOR / ZGRTYPE 都支援萬用字元 '*' (代表「不限」)，比對邏輯採「精確優先、萬用其次」：先試「精確 FEVOR + 精確 ZGRTYPE」，找不到再試「精確 FEVOR + *」，再試「* + 精確 ZGRTYPE」，最後才試「* + *」，命中的第一組就是最終規則；ZEN04_SEQ 的 Key 用的是實際命中的那組值 (如果是靠萬用字元命中，Key 就用 '*')，確保「不限」的規則底下所有情況共用同一組流水號，不會因為每個不同的 FEVOR/ZGRTYPE 組合各自開一份流水號而變得瑣碎。

跨程式讀寫全域變數的技巧 (ASSIGN 動態存取)：原始案例裡有一行很有教學價值的程式碼：

```
DATA: lcfnam1(30) VALUE '(SAPLCOK01)AFPOD-WEMPF'.
FIELD-SYMBOLS: <fs>.
ASSIGN (lcfnam1) TO <fs>.
```

WEMPF (收貨對象) 不是 CO_ZF_NUMBER_GET 自己的參數，而是另一個 Function Group (COK01，工單維護的主力程式) 用 TABLES: afpod. 宣告的全域工作區欄位。用 ASSIGN ('(程式名)結構-欄位') TO <fs> 這種動態語法，可以跨程式讀取 (甚至寫入) 別支程式的全域資料——這是很多老 SAP 系統客製化常見的手法：真實交易流程裡，SAPLCOK01 早已在使用者操作 C001/COR1 的過程中把 AFPOD-WEMPF 填好，CO_ZF_NUMBER_GET 被呼叫時，這個全域早就有正確的值可以讀。

⚠ 這個技巧的前提：目標程式必須已經被載入記憶體——如果 SAPLCOK01 / SAPLCOZF 在目前這個 ABAP session 裡一次都還沒被呼叫過 (沒有任何 CALL FUNCTION 命中過該 Function Group)，ASSIGN 會安靜地失敗 (sy-subrc <> 0)，不會有任何錯誤訊息或 dump——這正是本題端對端測試第一次失敗的根因：獨立寫

的測試程式直接呼叫 `CO_ZF_NUMBER_GET`，但因為從沒呼叫過 `COK01` 群組的任何 FM，`ASSIGN` (`'(SAPLCOK01)AFPOD-WEMPF'`) 讀不到東西，`ZGRTYPE` 永遠是空白，規則表永遠比對失敗，客製化邏輯完全沒有介入的機會，三次呼叫都拿到標準取號結果——修法是先呼叫一支該群組裡安全、唯讀、無副作用的 FM（本題用 `CO_K01_GET_HEADER`，只有 `EXPORTING` 參數、呼叫端可以完全不接收）當「暖身呼叫」，讓程式群組被載入記憶體後，`ASSIGN` 才會成功。

⚠ 另一個重要教訓：**Enhancement 啟用生效**，跟「使用者 **GUI Session** 看不看得到最新版本」是兩件事——用 ADT 把 Enhancement 內容改好、啟用、`sap_inactive_objects` 確認無殘留，這只代表系統裡的正式版本已經更新；但如果使用者的 SAP GUI 在這之前就已經在同一個登入 Session 裡進過相關交易、載入過 `SAPLCOZF` 的「已產生 (generated)」版本，這個 Session 剩下的生命週期都會繼續用載入當下那個舊版本，即使背後的原始碼已經換了。本題實測踩到：先寫了一個「不論任何條件、寫死測試值後 `EXIT`」的診斷版本驗證插入點本身有沒有作用，改回正式邏輯、重新啟用之後，使用者馬上用 `C001` 建立一張真實工單，結果訂單號碼變成了診斷版本寫死的測試字串——不是正式版本沒生效，是使用者那個 GUI Session 記憶體裡還在用診斷版本產生的程式。**解法**是使用者完全登出再重新登入（或開一個新的 **Mode/Session**），全新 Session 第一次載入該程式群組時，才會抓到最新已啟用的版本。這個現象跟 ADT `programrun`（每次呼叫視同全新、無狀態的 Session）不會踩到，只有長時間開著的互動式 GUI Session 才會出現，出題/教學時務必提醒學員這一點，避免誤判「改了沒生效」。

建立方式 (GUI-only，ADT 無法建立空殼，但寫程式碼與啟用可以走 ADT)：跟 en02 的 `ZX Include` 一樣，Source Code Plugin (ADT 物件型別 `ENHO/XHH`) 綁定「這支 FM 內部某個位置的隱式增強點 ID」，這個位置 ID 只有在 SE37/SE38 編輯器裡實際瀏覽過 Enhancement Operations 才會產生，沒有對應的 ADT 建立 API（實測直接 `POST /sap/bc/adt/enhancements/enhoxhh` 一律報 `System expected the element enho:enhancement`，無法用猜測的 XML 結構繞過）。正確流程：

1. **SE37** → Function Module `CO_ZF_NUMBER_GET` → Display
2. **Edit** → **Enhancement Operations** → **Show Implicit Enhancement Options**（畫面上出現插入點圖示）
3. 找到參數宣告結束、第一行可執程式碼之前的插入點 → **Create** → **Enhancement Implementation**
4. 填 Enhancement Implementation 名稱（本題 `ZEN04_ORDER_NO_PLUGIN`）、套件（`$TMP`）
5. 系統生成空的 `ENHANCEMENT 1 . ... ENDENHANCEMENT` 骨架，內容留空、存檔
6. 之後 Claude 可以正常用 ADT (`GET/PUT` + `LOCK/UNLOCK` + activation API，流程同 `.claude/rules/sap-adt-mcp.md` 第 5 節的殘留鎖 workaround) 讀寫這個物件的內容，**不需要每次都重新走 GUI**

寫入時的一個格式陷阱：PUT 進去的原始碼，`ENHANCEMENT` 這一行不能自己補上 **Enhancement Implementation** 的名稱——GET 回來的空骨架長這樣：`ENHANCEMENT 1 .`（數字後面兩個空白直接接句點，沒有名稱），如果自己「補齊」寫成 `ENHANCEMENT 1 ZEN04_ORDER_NO_PLUGIN.` 送出去，會收到 `ExceptionResourceScanDuringSaveFailure` ("Scan of resource failed")——名稱是物件中繼資料層級管理的，不能寫進原始碼本文，照抄 GET 回來的骨架格式就對了。

學習目標

- 能講出 Implicit 與 Explicit Enhancement Point 的差異：Implicit 到處都有、不需要原開發者宣告，但只能插入程式碼不能改介面
- 能講出「插在最前面 + 條件成立時 `EXIT`」這個組合為什麼能讓客製化邏輯完全取代標準邏輯，而不是被蓋掉
- 能設計一個「啟用總開關表 + 規則表 + 流水號進度表」三層架構，理解為什麼要把「要不要客製化」「客製化成什麼樣」「目前流水號多少」拆成三張表而不是塞在一起

- 能講出萬用字元 (*) 比對「精確優先、萬用其次」的設計方式，以及為什麼共用流水號要用「實際命中的規則」當 Key，而不是原始輸入值
- 能講出動態 **ASSIGN** 跨程式讀寫全域變數的技巧與前提條件（目標程式必須已載入記憶體），知道獨立測試這類邏輯時要先做「暖身呼叫」
- 理解「Enhancement 已啟用」跟「使用者 GUI Session 看到最新版本」是兩件事，知道互動式 Session 需要重新登入才會抓到新版本
- 能講出 Source Code Plugin (**ENHO/XHH**) 的建立流程是 GUI-only (SE37 Enhancement Operations → Create)，但骨架建好之後的讀寫可以走 ADT，且原始碼裡不能自己補 Enhancement 名稱

事前準備 (已於本系統 client 130 實際完成，非假設)

1. 三張自訂表 (\$TMP) : ZEN04_PLTAUART (WERKS+AUART 為 Key，無資料欄位)、ZEN04_RULE (WERKS+AUART+FEVOR+ZGRTYPE 為 Key，LEADCODE+STNUM 為資料欄位)、ZEN04_SEQ (WERKS+AUART+FEVOR+ZGRTYPE+LEADCODE+ZYEAR+ZMONTH 為 Key，NUMNO+AUFNR 為資料欄位)，均已建立並啟用。(2026-07-30 補課：全部欄位改為引用 Data Element，不再直接用 Builtin Type ——WERKS→標準 DE WERKS_D、AUART→標準 DE AUFART、FEVOR→標準 DE FEVOR、AUFNR→標準 DE AUFNR；ZGRTYPE/LEADCODE/STNUM/ZYEAR/ZMONTH/NUMNO 六個課程自訂欄位沒有語意相符的標準 DE，各自新建一組 Domain+DE (ZEN04_ZGRTYPE/ZEN04_LEADCOD/ZEN04_STNUM/ZEN04_ZYEAR/ZEN04_ZMONTH/ZEN04_NUMNO)，沿用 en08 案例一「能引用 Data Element 就不要用 Builtin Type」的 DDIC 慣例。既有資料在型別轉換後確認保留，未受影響。)
2. ZCL_EN04_ORDER_NUMBERING (\$TMP) : 核心編號邏輯類別，get_custom_order_number 方法依三張表算出號碼，支援 FEVOR/ZGRTYPE 萬用字元「精確優先、萬用其次」比對，已建立並啟用。
3. ZEN04_ORDER_NO_PLUGIN : 使用者於 SE37 對 CO_ZF_NUMBER_GET 建立 Implicit Enhancement (Source Code Plugin)，插入點位置 \FU:CO_ZF_NUMBER_GET\SE:BEGIN\EI (Function Module 最前面)，Claude 用 ADT 寫入呼叫 ZCL_EN04_ORDER_NUMBERING 的程式碼並啟用。
4. 端對端驗證 (兩組情境，均已實測成功) :
 - 安全測試組合 (WERKS='ZZ99' / AUART='ZE04'，確認系統裡不存在的組合) :
ZR_EN04_ORDER_NUMBER_DEMO (\$TMP) 用 programrun 無頭執行，真實輸出：

`` 第 1 次呼叫 (啟用組合 ZZ99/ZE04) CAUFVD_EXP-AUFNR = TR2670001 第 2 次呼叫 (啟用組合 ZZ99/ZE04) CAUFVD_EXP-AUFNR = TR2670002 第 3 次呼叫 (未啟用組合 9999/9999) CAUFVD_EXP-AUFNR = 1001805 ` (LEADCODE='TR'、STNUM='0001'；第 3 次未登記組合正確 fall through 走標準 Number Range 取號)

- 真實廠別/製令類型 + 萬用字元規則 (WERKS='1011' [東捷南港廠] / AUART='PP71'，使用者確認為教學專用製令類型；FEVOR='*' / ZGRTYPE='*' / LEADCODE='PP' / STNUM='0001') :
ZR_EN04_TEST_1011_PP71 (\$TMP) 用完全不同的兩組 FEVOR/WEMPF (XYZ/ANYONE0001 與 999/DIFFERENT9) 測試，真實輸出：

`` 第 1 次呼叫 (FEVOR=XYZ / WEMPF=ANYONE0001) CAUFVD_EXP-AUFNR = PP2670001 第 2 次呼叫 (FEVOR=999 / WEMPF=DIFFERENT9) CAUFVD_EXP-AUFNR = PP2670002 ` 證實萬用字元規則不限 FEVOR/ZGRTYPE` 為何值都能命中、且正確共用同一組流水號。

- 使用者亦透過真實 C001 (Plant 1011 / Order Type PP71) 交易建單驗證 Enhancement 對真實工單建立確實生效。

題目需求

1. 解釋「插在最前面 + 條件成立時 EXIT」的機制，說明為什麼這樣寫客製化邏輯不會被後面的標準程式碼覆蓋掉，並指出如果拿掉 EXIT 會發生什麼事。
2. 設計三層表格架構的理由：為什麼不把「要不要客製化」和「客製化成什麼樣」合併成一張表？如果之後要暫時停用某個廠別/製令類型的客製化（不想砍掉規則設定），哪張表要動？
3. 萬用字元比對邏輯：畫出「精確 FEVOR + 精確 ZGRTYPE」「精確 + *」「* + 精確」「* + *」四種情況的比對順序圖，說明為什麼要「精確優先」而不是隨便挑一個命中的規則。
4. ASSIGN 動態存取的風險：解釋為什麼獨立寫測試程式呼叫 CO_ZF_NUMBER_GET 第一次會拿到標準取號結果，而不是預期的自訂格式，並說明修法。
5. GUI Session 快取現象：解釋為什麼 Enhancement 已經用 ADT 啟用成功，使用者馬上用 CO01 建單卻可能看到舊版本的行為，該怎麼解決。

參考答案

EXIT 機制：Implicit Enhancement 插在 FM 最前面，執行順序上先於所有標準程式碼；當客製化規則命中（兩層安全閘都通過）時，程式碼設定好 CAUFVD_EXP-AUFNR 之後立刻 EXIT（效果同 RETURN），FM 的執行流程直接在這裡結束，後面的 DATA: 宣告、CALL FUNCTION 'NUMBER_GET_NEXT'、CASE OBJECT. 判斷完全不會被執行到——不是「先跑完標準邏輯、客製化值再被蓋掉」，是「客製化命中時標準邏輯根本沒機會跑」。如果拿掉 EXIT，程式會繼續往下執行標準的 NUMBER_GET_NEXT 呼叫與 CASE OBJECT. 判斷，CAUFVD-AUFNR / CAUFVD_EXP-AUFNR 會被標準邏輯的結果覆寫回去，客製化編號就完全失效。

三層表格架構：ZEN04_PLTAUART（開關）、ZEN04_RULE（規則內容）、ZEN04_SEQ（流水號狀態）職責不同、變化頻率也不同——開關可能今天開明天關（暫停某個組合的客製化，不代表規則本身要刪掉，之後隨時可能重新開啟）；規則內容一旦設定通常穩定不太變動；流水號狀態則是每次呼叫都會變的高頻資料。如果合併成一張表，「暫停客製化」就得整列刪除、之後要重開又得把 LEADCODE/STNUM 重新輸入一次，容易出錯；分開之後，暫停/啟用只是 ZEN04_PLTAUART 增刪一行，ZEN04_RULE 的規則內容完全不受影響——要暫停客製化，只動 ZEN04_PLTAUART，不動 ZEN04_RULE。

萬用字元比對優先序：

```
精確 FEVOR + 精確 ZGRTYPE  ← 最先嘗試（最具體）
    ↓ 找不到
精確 FEVOR + '*'
    ↓ 找不到
'*' + 精確 ZGRTYPE
    ↓ 找不到
'*' + '*'                  ← 最後嘗試（最不具體，等於「不限」）
```

要「精確優先」是因為業務上常見「大部分情況用同一套規則，少數例外要特別處理」的需求——如果反過來讓萬用字元規則優先命中，那些為特殊情況設計的精確規則就永遠沒有機會生效（因為萬用規則會先攔截所有請求）。這是很多規則引擎（防火牆規則、路由表、CSS 選擇器優先權）共通的設計原則：越具體的規則，優先權越高。

ASSIGN 失敗的風險與修法：ASSIGN ('(SAPLCOK01)AFPOD-WEMPF') TO <fs> 能不能成功讀到值，前提是 SAPLCOK01 這支程式已經在目前的 ABAP session 裡被載入記憶體（有任何一次 CALL FUNCTION 命中過 COK01 這個 Function Group）。獨立寫的測試程式如果只呼叫 CO_ZF_NUMBER_GET（屬於 COZF 群組），從未觸碰到 COK01 群組的任何 FM，這個 ASSIGN 會安靜地失敗（sy-subrc <> 0，沒有錯誤訊息或 dump），ZGRTYPE 永遠是初始值（空白），規則表永遠比對失敗，客製化邏輯完全沒有介入的機會。修法是在測試程式裡先呼叫

一支 COK01 群組裡安全、唯讀、無副作用的 FM 當「暖身呼叫」(本題用 CO_K01_GET_HEADER , 只有 EXPORTING 參數, 呼叫端可以完全不接收任何回傳值), 讓程式群組被載入之後, ASSIGN 才會成功——這個技巧在真實交易流程 (CO01/COR1) 裡不需要, 因為使用者操作過程中 SAPLCOK01 早就被載入了, 只有「獨立、繞過正常交易流程」的測試才會踩到這個前提條件。

GUI Session 快取現象：SAP 的 ABAP 執行環境會把已經載入過的程式 (Generated Code) 留在該登入 Session 的記憶體裡, 直到 Session 結束。如果使用者在 Claude 修正並重新啟用 Enhancement 之前, 就已經在同一個登入 Session 裡進過相關交易、載入過 SAPLCOZF, 那麼即使背後的 Enhancement 原始碼已經更新、啟用成功, 這個 Session 剩下的生命週期都會繼續使用當初載入時的舊版本, 直接影響到的就是使用者馬上用 CO01 建單卻得到不符預期的結果 (本題實測就踩到: 拿到的是稍早診斷版本寫死的測試字串, 而非正式邏輯應該產生的格式)。解法很單純: 使用者完全登出再重新登入 (或開一個全新的 Mode/Session), 新 Session 第一次載入該程式群組時就會抓到最新已啟用的版本; ADT 的 programrun 每次呼叫都是全新、無狀態的執行環境, 不會踩到這個問題, 所以拿它做端對端驗證時看到的都是即時生效的結果, 這也是為什麼「先用 headless 測試驗證邏輯正確, 再請使用者在 GUI 重新登入後驗證」是比較穩妥的驗收順序。

思考題

1. 這題的安全閘設計 (ZEN04_PLTAUART 存在性判斷) 刻意選用了系統裡保證不存在的廠別/製令類型組合 (ZZ99/ZE04) 做第一輪測試, 後來才改用真實的 1011/PP71。如果一開始就直接拿真實組合測試、且客製化邏輯有 bug (例如忘記檢查安全閘、無條件攔截所有 OBJECT='AUFTRAG' 的呼叫), 會造成什麼後果? (提示: 本題端對端測試過程中, 真實發生過一次「診斷版本沒有任何條件判斷、寫死測試值後 EXIT」的情境, 被使用者拿真實 CO01 交易測到, 系統裡因此真的多了一張訂單號碼是垃圾字串的正式工單——這正是為什麼修改標準物件的行為時, 一定要先用保證不存在的假資料驗證機制本身沒問題, 最後才切換到真實資料)
2. ZEN04_SEQ 的 Key 用的是「實際命中規則」的 FEVOR/ZGRTYPE (可能是 '*'), 而不是呼叫當下的原始 iv_fevor/iv_wempf。如果反過來用原始值當 Key, 在「規則是萬用字元、但呼叫時傳進來的 FEVOR/WEMPF 每次都不一樣」的情境下, 會發生什麼問題? (提示: 每個不同的 FEVOR/WEMPF 組合都會各自開一份 ZEN04_SEQ 列, 流水號變成各算各的, STNUM 起始值會被重複使用很多次, 完全達不到「同一個 Lead Code 底下流水號要連續」的設計目的)
3. 為什麼 Implicit Enhancement Point 的建立 (SE37 裡雙擊/建立那一步) 沒有對應的 ADT API, 但骨架建好之後的原始碼讀寫、啟用卻可以走 ADT? 這跟 en02 學到的「ZX 開頭 Include 保留給 Exit Function Group、CMOD 裡雙擊才會生成」是不是同一種限制? (提示: 兩者都是「某個位置/身分需要在 GUI 編輯器裡實際操作過才會產生系統內部的技術 ID (隱式增強點位置、Exit Include 保留命名空間), 這個 ID 沒有對應的 REST 資源可以憑空 POST 出來; 但 ID 一旦存在, 後續的內容維護就是標準的物件讀寫, ADT 完全可以勝任」是同一類限制)

答案

ZEN04_PLTAUART / ZEN04_RULE / ZEN04_SEQ (三張自訂表, \$TMP, 快照見 zen04_pltauart.tabl.abap / zen04_rule.tabl.abap / zen04_seq.tabl.abap)、ZCL_EN04_ORDER_NUMBERING (核心編號邏輯類別, \$TMP, 快照 zcl_en04_order_numbering.clas.abap, 支援 FEVOR/ZGRTYPE 萬用字元精確優先比對)、ZEN04_ORDER_NO_PLUGIN (Implicit Enhancement / Source Code Plugin, 插在 CO_ZF_NUMBER_GET 最前面, 快照 zen04_order_no_plugin.enho.abap) 均已建立並啟用 (sap_inactive_objects 確認無殘留未啟用版本)。驗證程式 ZR_EN04_ORDER_NUMBER_DEMO (安全測試組合 ZZ99/ZE04) 與 ZR_EN04_TEST_1011_PP71 (真實廠別/製令類型 + 萬用字元, 1011/PP71) 均已用 programrun 無頭執行成功, 並經使用者用真實 CO01 交易端對端驗證 Enhancement 對真實工單建立確實生效。三層表格架構設計理由、萬用字元比對優先序、ASSIGN 動態存取前提條件、GUI Session 快取現象見本題內文與 Lecture。